# `std::move` is for the last time you use a variable.

## Not for returns.

```
std::string s = read();
names.push_back(std::move(s));
// s is finished
```

Knowing when to write it matters more than knowing what it is.

swipe →

# Use it to hand something over

```
names.push_back(std::move(s));


struct Person {
    std::string name;
    Person(std::string n)
        : name(std::move(n)) {}
};
```

- into a container
- into a member, from a by-value parameter
- into a `unique_ptr` or another owner

In each case the variable is not needed afterwards, so stealing its contents is free.

# Do not use it on a return

```
T make() {
    T x;
    return std::move(x);    // worse
}

T make() {
    T x;
    return x;               // better
}
```

A plain `return x;` already moves, or builds `x` in the caller with no move at all.

Writing `std::move` there blocks that. GCC warns about it with `-Wpessimizing-move`.

# After the move

```
std::string s = "a long string...";
names.push_back(std::move(s));

s.size();   // 0 here, but not guaranteed
```

**A moved-from object is valid, but its value is unspecified.**

- safe: destroy it, or assign a new value
- not safe: depend on what it contains

# Moving a `const` copies

```
const T c;
T d = std::move(c);   // copy

T e;
T f = std::move(e);   // move
```

A `const` object cannot be emptied, so the move constructor does not match.

I checked it with a type that prints "copy" and "move". The compiler silently picks the copy.

# When to write it

| SITUATION | STD::MOVE? |
| --- | --- |
| last use, handing it on | **yes** |
| filling a member from a parameter | **yes** |
| `return x;` | **no** |
| the object is `const` | **no** |
| you use it again afterwards | **no** |

**Have you ever written `return std::move(x);`? Be honest.**